# EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

## BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

# WEEK 1

## Business Needs, Software Needs and Object Technology for Business

| | |
|---|---|
| **PROGRAMME** | BSc Information Communication and Technology |
| **COURSE CODE / YEAR** | BICT 3202 · Year 3 |
| **LECTURER** | Francis Fweta — ICT Lecturer |
| **DELIVERY** | 2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning |

*Prepared by Francis Fweta · Exploits University · Week 1 of 16*

# Contents

1. Week 1 Overview 3
2. Learning Outcomes 3
3. Introduction: Why Do We Need OOAD? 4
4. Business Needs 5
5. Stakeholders 6
6. Aspects of Business Software Needs 8
7. Object Technology for Business 12
8. Case Study & Worked Example 17
9. Practical & Tutorial Activities 18
10. Independent Learning (10 Hours) 18
11. Common Student Mistakes 19
12. Week 1 Summary 19
13. Revision Questions 20
14. References and Further Reading 21

# 1. Week 1 Overview

This week covers the first three topics listed under the module's Topics of Study: **Business Needs**, **Aspects of Business Software Needs** and **Object Technology for Business**.

These topics are deliberately placed at the beginning of OOAD. Before students can create classes, objects, use cases or UML diagrams, they must first understand **why** an organisation needs software and **what problem** the software is supposed to solve.

> **COMMON MISCONCEPTION**
>
> "Object-Oriented Analysis and Design starts by drawing a class diagram." — **It does not.**
>
> A good OOAD process starts by understanding the problem, business objectives, stakeholders and requirements. Requirements engineering — discovering, analysing, documenting, validating and managing what a system must accomplish — follows ISO/IEC/IEEE 29148:2018, the current published edition as of 2026.

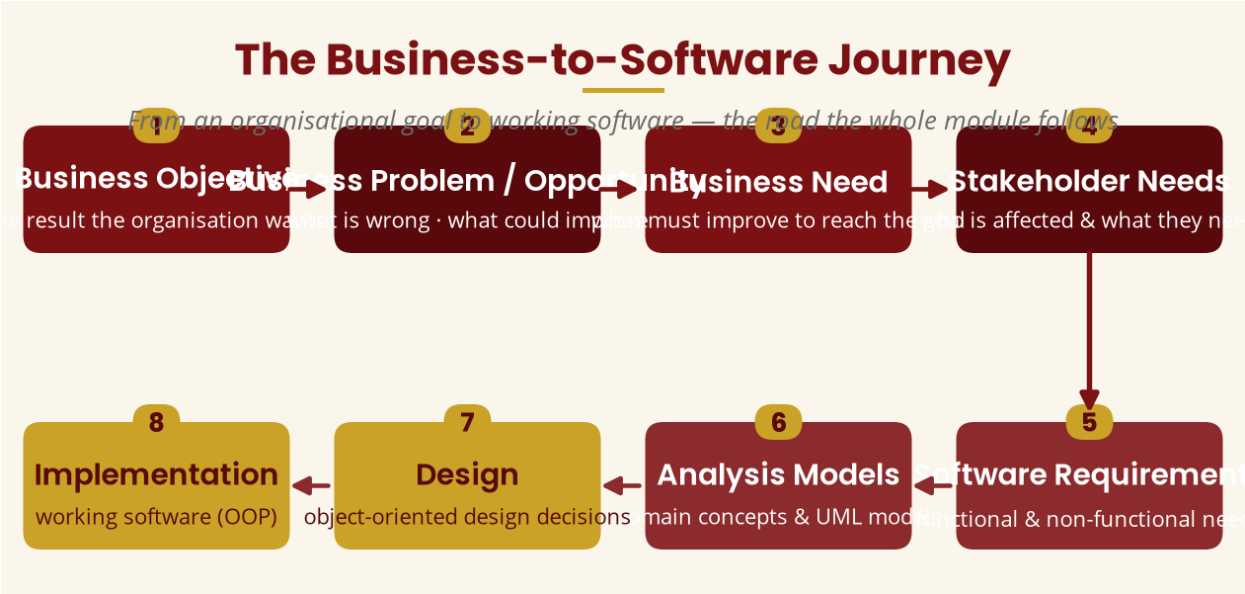


*Figure 1 — The logical journey: business problem → business need → software need → requirements → analysis → models → design → implementation*

# 2. Learning Outcomes

By the end of this week, students should be able to:

- Define a business need.
- Explain why organisations develop or acquire software systems.
- Distinguish between a business problem, a business need, a requirement and a software solution.
- Identify stakeholders associated with a business system.
- Explain functional and non-functional software needs.
- Explain the relationship between business objectives and software requirements.
- Describe the basic idea behind object technology and explain what an object is.

- Explain the relationship between objects, state and behaviour.
- Explain why real-world entities are useful when analysing software systems.
- Describe major advantages and limitations of object-oriented approaches.
- Apply these concepts to a simple real-world business case.

## 3. Introduction: Why Do We Need OOAD?

Imagine that a university tells a software developer: "We need a new student management system." The developer immediately starts programming — a login page, a student table, a lecturer table, a payment page, an examination page and a dashboard.

After six months the university tests the system and says: "This isn't what we asked for." The developer replies: "But you told us to build a student management system." The university responds: "Yes, but students should register courses **before** paying fees. Lecturers should approve registration. Students should only register for courses offered by their programme. Finance should see outstanding balances. Students should receive notifications. And we need to prevent registration if prerequisites haven't been met."

**THE LESSON**

The problem was not programming ability — it was **insufficient analysis of the business needs before designing the software**. This is one of the fundamental reasons OOAD exists.

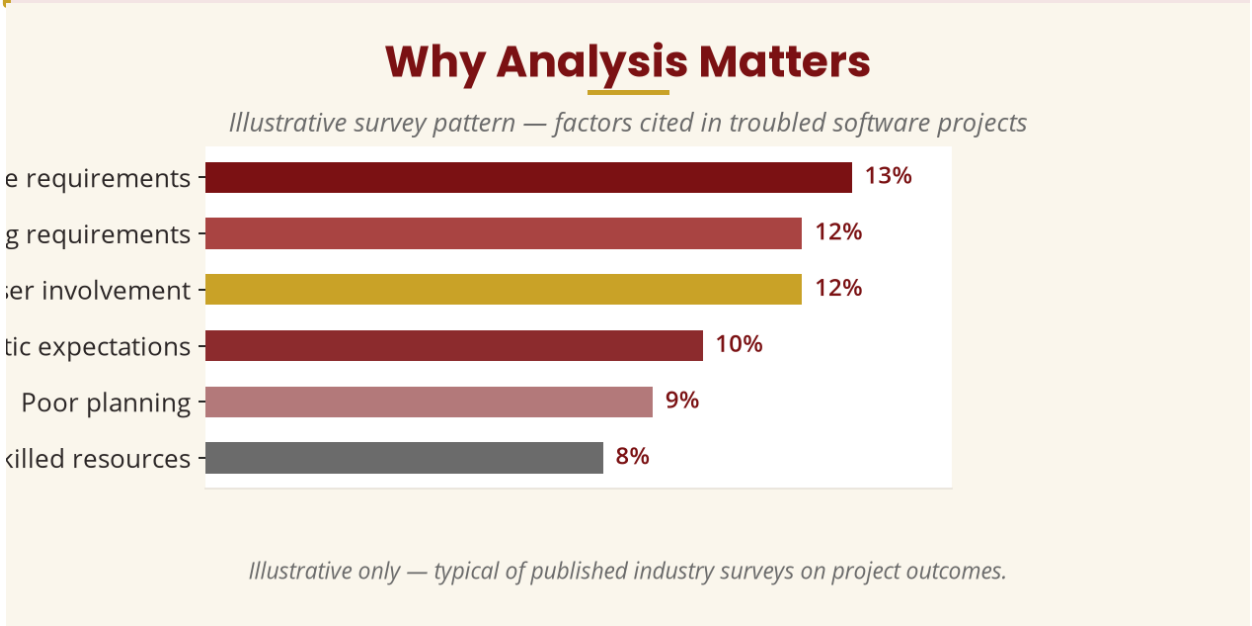


*Figure 2 — Illustrative survey pattern: incomplete or misunderstood requirements are the leading contributor to troubled software projects*

# 4. Business Needs

## 4.1 What is a Business Need?

A business need is a problem, opportunity, objective or desired improvement that an organisation must address in order to achieve its goals.

> **IN SIMPLE LANGUAGE**
>
> A business need explains **why** an organisation needs to change something.

For example, a university currently records student registration using paper forms and students spend hours waiting in queues. The university therefore identifies a need: *"Reduce the time required for student registration and improve the accuracy of registration records."*

Notice that the business need is **not yet software**. The university has not said "We need a Java application" — it has identified a business problem and a desired improvement.

## 4.2 Business Problem vs Business Need

A **business problem** describes an undesirable current situation. A **business need** describes what the organisation must achieve or improve in response to that situation.

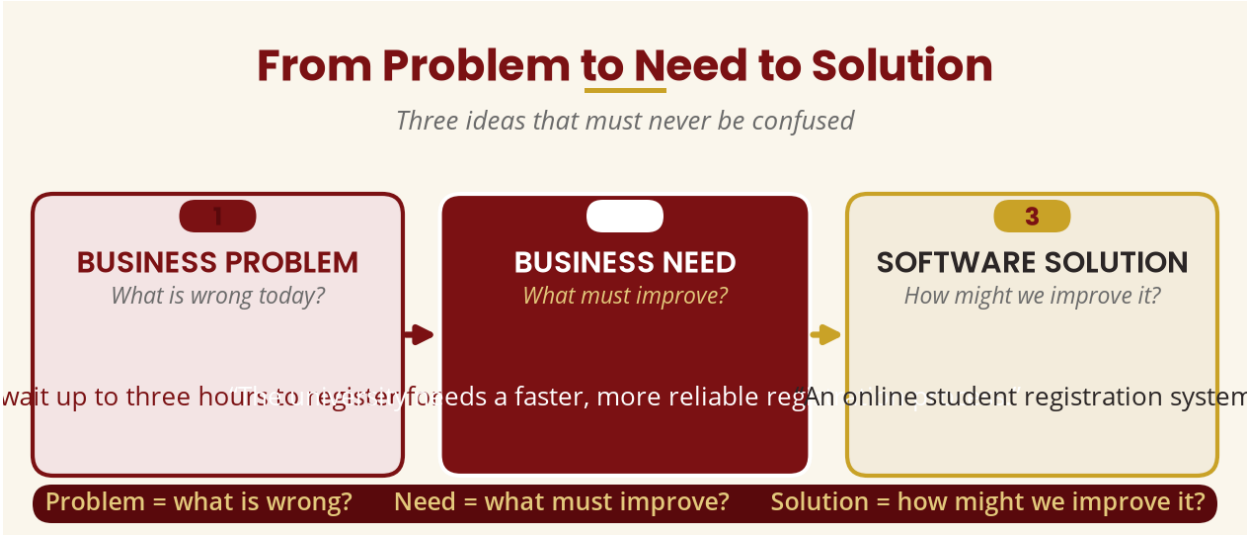


*Figure 3 — Problem, need and solution are three distinct ideas*

## 4.3 Business Needs Are Not Automatically Software Needs

Suppose a hospital says: "Patients wait too long before seeing a doctor." Is the solution automatically "build hospital software"? No. There could be several causes — too few doctors, patients incorrectly assigned to queues, slow registration, hard-to-locate patient files, inefficient scheduling, or emergency cases mixed with normal appointments.

Software may solve **some** of these problems, but not necessarily all of them. An analyst must investigate the underlying business problem before proposing technology.

### 4.4 Business Objectives

A business objective is a specific result that an organisation wants to achieve. Examples include: increase sales, reduce operating costs, improve customer satisfaction, reduce processing time, improve data accuracy, comply with regulations, increase security, expand services, and improve decision-making.

> **EXAMPLE — BANK**
>
> Objective: *reduce the average time to open a customer account from 45 minutes to 10 minutes.*
>
> That objective leads to a business need (a faster account-opening process), which leads to software requirements: customers submit information electronically, staff verify identity digitally, the system validates required information, and the system notifies customers about application status.

### 4.5 The Business-to-Software Chain

This chain is the backbone of the entire 16-week module:

```
BUSINESS OBJECTIVE
    ↓
BUSINESS PROBLEM / OPPORTUNITY
    ↓
BUSINESS NEED
    ↓
STAKEHOLDER NEEDS
    ↓
SYSTEM / SOFTWARE REQUIREMENTS
    ↓
ANALYSIS MODELS
    ↓
DESIGN
    ↓
IMPLEMENTATION
```

## 5. Stakeholders

### 5.1 What is a Stakeholder?

A stakeholder is a person, group or organisation that has an interest in, is affected by, influences, or interacts with a system.

For a university registration system, possible stakeholders include: students, lecturers, registry officers, finance officers, department heads, administrators, ICT staff, university management and external regulators.

Not every stakeholder necessarily uses the software directly — the Vice Chancellor may never log into the registration system but is still affected by its performance and reports.

## 5.2 Primary and Secondary Stakeholders

**Primary stakeholders** directly interact with the system or are directly affected by it (Student, Lecturer, Registrar, Finance Officer). **Secondary stakeholders** may not directly use the system but have an interest in its outcomes (University Management, Auditors, Regulators, ICT Administrators).

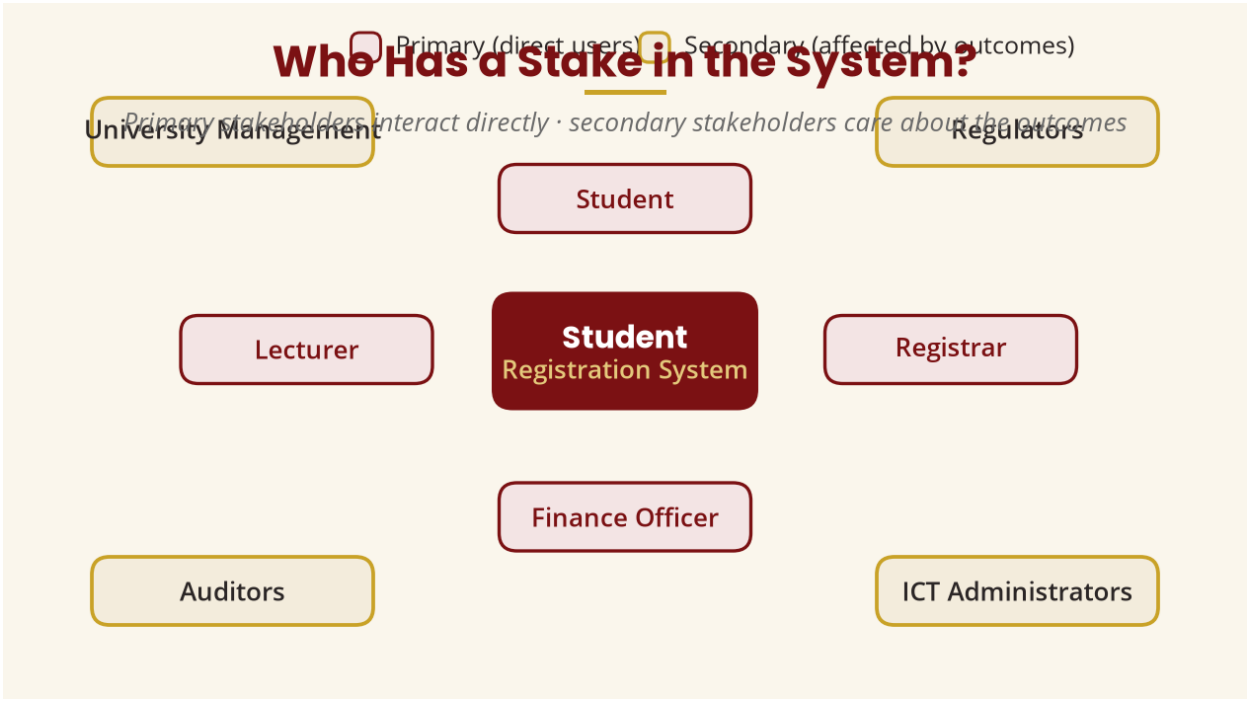


*Figure 4 — Stakeholder map for a student registration system*

## 5.3 Why Stakeholders Matter in OOAD

If developers interview only students, they hear "We need online registration." But Finance might say "Students with outstanding fees must not register," Registry might say "Students must meet programme requirements," Lecturers might say "Some courses require approval," ICT might say "Only authorised staff access administration," and Management might say "We need registration statistics reports."

**KEY POINT**

If the analyst ignores stakeholders, **important requirements will be missed**.

| Stakeholder | Interest / Need |
|---|---|
| Student | Register courses |
| Lecturer | Approve or monitor registration |
| Registrar | Maintain academic registration records |
| Finance Officer | Verify financial eligibility |
| Department Head | Monitor programme registration |
| ICT Administrator | Maintain system and security |

| Stakeholder | Interest / Need |
|---|---|
| Management | Obtain reports |

# 6. Aspects of Business Software Needs

A business does not simply need "software" — it usually needs software that satisfies **several categories** of needs. The most important distinction is functional versus non-functional needs.

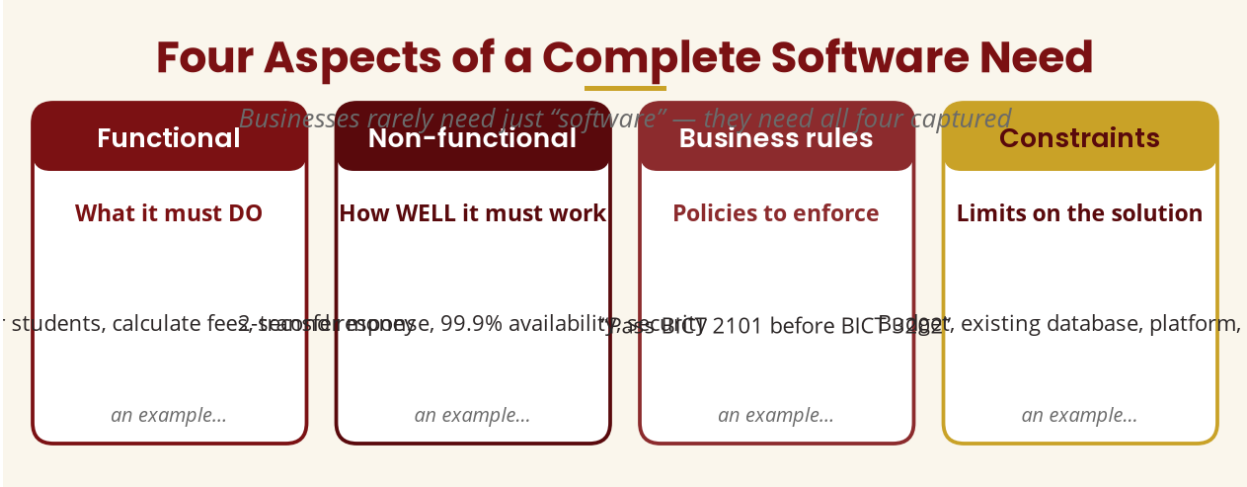


*Figure 5 — Four aspects of a complete software need*

## 6.1 Functional Software Needs

A functional requirement describes **what the system should do** — think **FUNCTION = ACTION**. Examples: register students, calculate fees, generate invoices, send notifications, reset passwords, generate examination reports.

> **EXAMPLE**
>
> For an online banking system: *"The system shall allow customers to transfer money between accounts."* — a functional requirement because it describes something the system must do.

## 6.2 Non-Functional Software Needs

A non-functional requirement describes a **quality, constraint or condition** under which the system must operate — think **NON-FUNCTIONAL = HOW WELL / UNDER WHAT CONDITIONS**. Examples: security, performance, availability, usability, reliability, scalability, maintainability and accessibility.

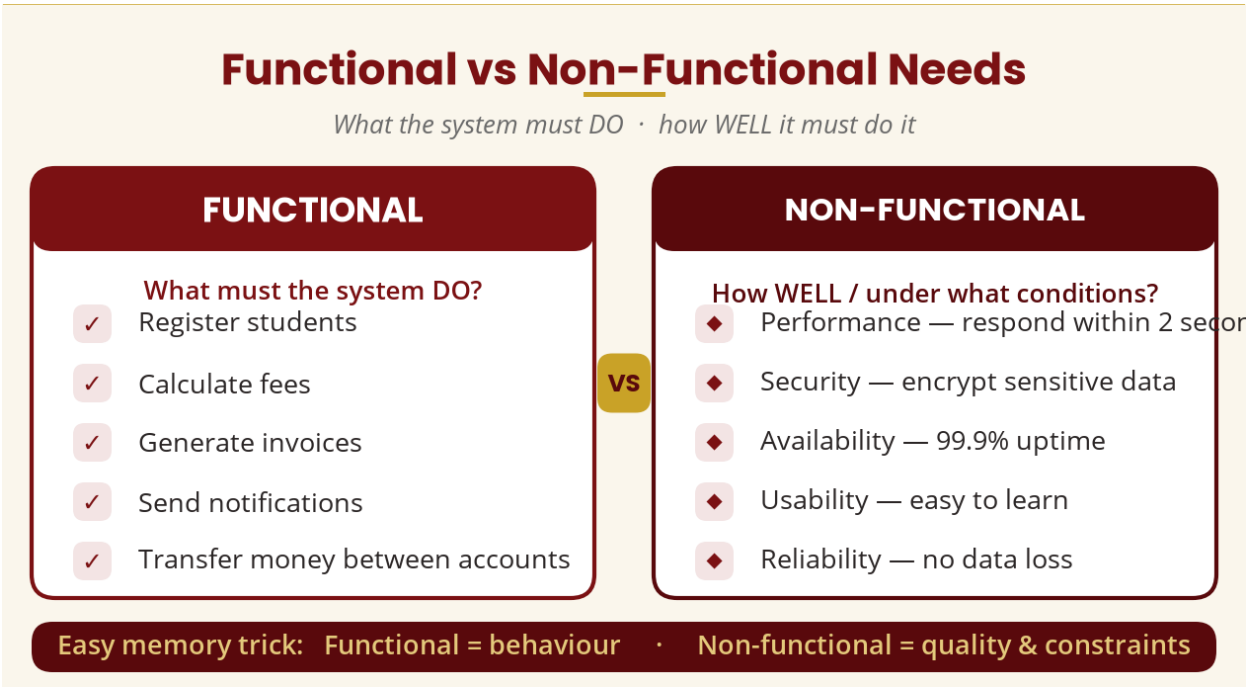


*Figure 6 — Functional (what) versus non-functional (how well)*

## 6.3 Functional vs Non-Functional — Comparison

| Aspect | Functional Requirement | Non-Functional Requirement |
|---|---|---|
| Main question | What must the system do? | What quality / constraint must it satisfy? |
| Example | Register student | Registration page loads within 2 seconds |
| Example | Generate invoice | Invoice available 99.9% of the time |
| Example | Transfer money | Transfers must be encrypted |
| Example | Search products | Search results appear quickly |

**Easy memory trick:** Functional = behaviour · Non-functional = quality / constraint.

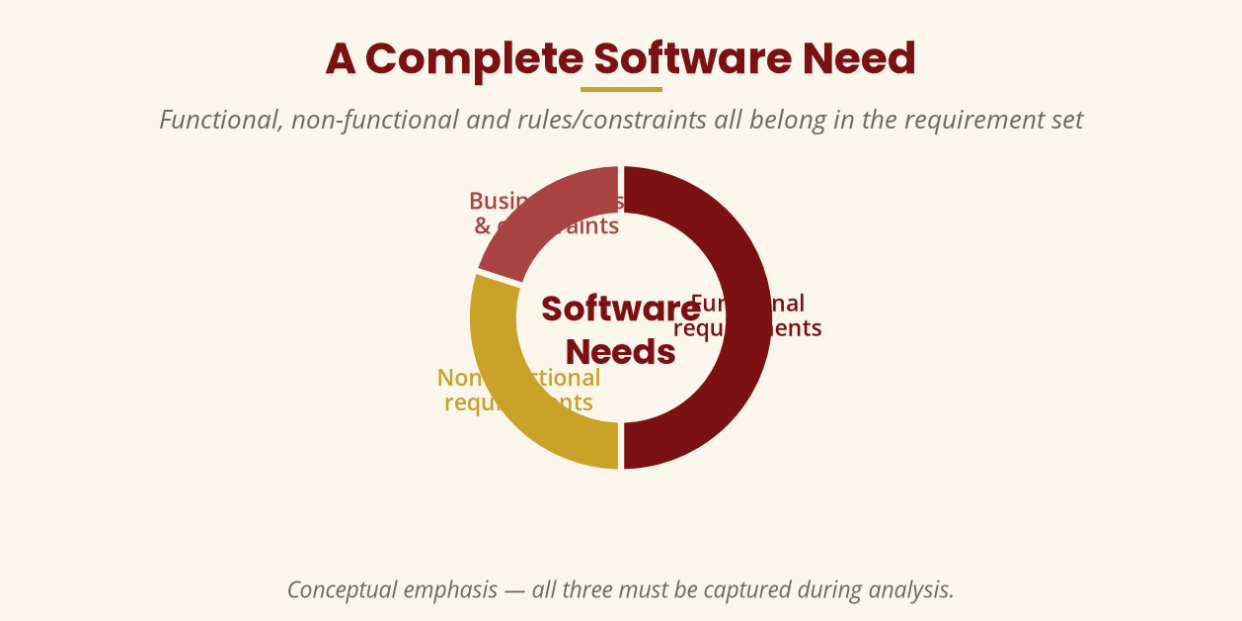


*Figure 7 — A complete requirement set captures functional, non-functional and rules/constraints*

## 6.4 Business Rules

A business rule is a policy, restriction or condition that governs how an organisation operates. The software must implement it. Examples:

• "A student may not register for a course unless the student has satisfied the course prerequisite."
• "A customer may withdraw a maximum of MWK 500,000 per day."
• "An employee cannot approve their own expense claim."

## 6.5 Business Rule vs Functional Requirement

**Business rule:** "Students must pass BICT 2101 before registering for BICT 3202."

**Functional requirement:** "The system shall check whether a student has passed the prerequisite before allowing registration for BICT 3202."

The business rule comes from the organisation; the functional requirement describes what the software must do to enforce or support that rule.

## 6.6 Constraints

Software also operates under **constraints** — limitations that restrict the possible solution: available budget, existing hardware, legal requirements, organisational policies, required platform, integration with an existing database, internet availability, and the deployment environment.

> **EXAMPLE**
>
> "The new system must work with our existing student database." — that is a constraint.

## 6.7 The Analyst's Job

An analyst does not simply ask "What software do you want?" Instead the analyst investigates:

1. What problem exists? 2. Why does it exist? 3. Who is affected?
4. What does the organisation want to achieve? 5. What processes currently exist?
6. What information is required? 7. What rules govern the process?
8. What should the new system do? 9. What constraints exist?
10. How will we know the solution is successful?

## 6.8 Requirements Elicitation

Requirements elicitation is the process of discovering and documenting stakeholder needs. ISO/IEC/IEEE 29148 describes it as using systematic techniques to proactively identify and document customer and end-user needs. Common techniques:

1. **Interviews** — talk directly to stakeholders ("How do you currently register students?")
2. **Questionnaires** — useful when many users must provide information
3. **Observation** — watch users perform their work
4. **Document analysis** — study forms, reports, policies, spreadsheets, databases, manuals
5. **Workshops** — stakeholders discuss requirements together
6. **Prototyping** — show a draft system and collect feedback


*Figure 8 — Six common requirements-elicitation techniques*

## 6.9 Worked Example: Student Registration

Suppose Exploits University wants a new registration system. The analyst works through seven steps:

**Step 1 — Business problem:** registration is paper-based, slow, difficult to monitor and vulnerable to data-entry errors.
**Step 2 — Business need:** a faster, more accurate and traceable registration process.
**Step 3 — Stakeholders:** students, lecturers, registry, finance, ICT, management.
**Step 4 — Functional requirements:** authenticate students, display available courses, check prerequisites, register/remove courses, submit for approval, generate reports.
**Step 5 — Non-functional requirements:** protect student information, be available during registration periods, respond quickly, support many simultaneous users, keep reliable records.
**Step 6 — Business rules:** "A student cannot register without satisfying the prerequisite."
**Step 7 — Candidate objects:** Student, Course, Lecturer, Registration, Programme, Payment,

Department.

> **NOTE**
>
> We have not designed the classes yet — we are simply beginning to identify important concepts in the problem domain. That distinction becomes critical in later weeks.

# 7. Object Technology for Business

## 7.1 What is Object Technology?

Object technology is an approach to analysing, designing and developing systems around **objects and their relationships**. Instead of viewing a system as a collection of procedures, object-oriented thinking asks: *What entities exist in this problem domain? What information does each entity have? What can each entity do? How do the entities interact?*

A software object combines related **state** (data) and **behaviour** (methods).

## 7.2 What is an Object?

An object is a distinct entity in a software system that has three parts: **identity**, **state** and **behaviour**.

**Identity** — the object can be distinguished from other objects (e.g. student ID ST001 vs ST002; both are Students, but they are separate objects).

**State** — the data describing its current condition (Student ID, Name, Programme, Year, Status). State can change: Status = Active may later become Graduated.

**Behaviour** — what the object can do (registerCourse(), dropCourse(), viewResults(), payFees()).

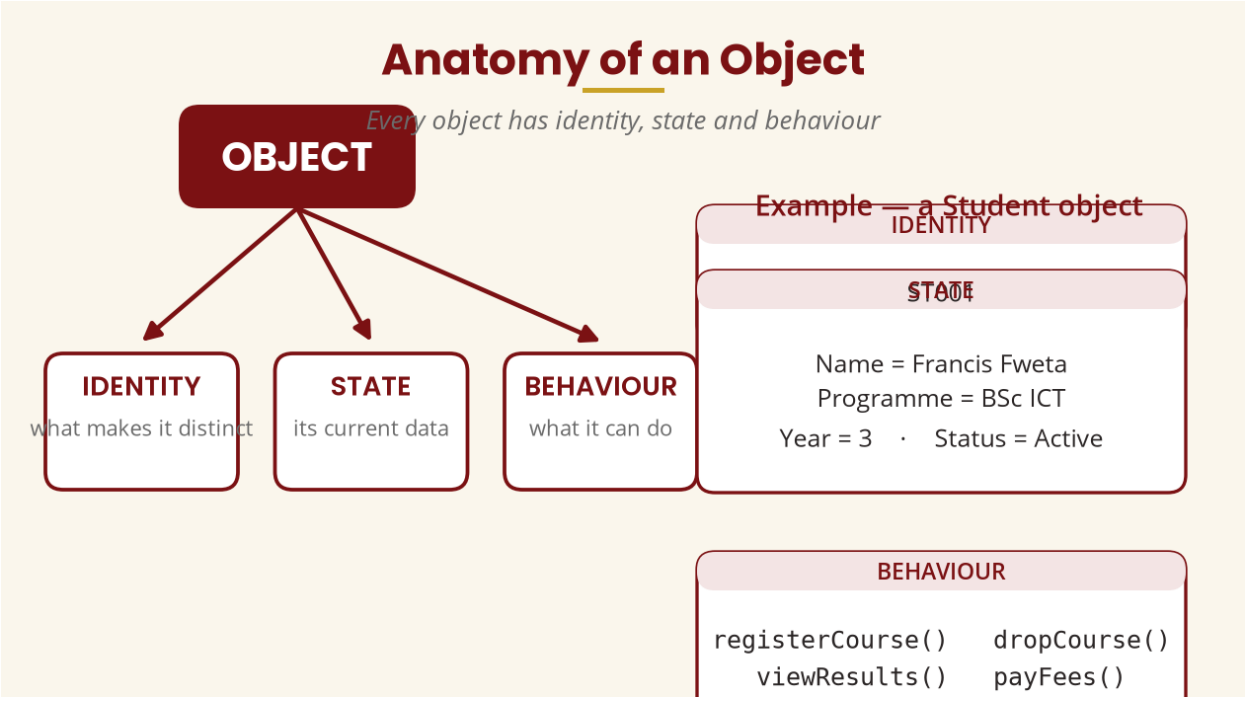


*Figure 9 — Identity, state and behaviour, with a Student example*

## 7.3 What is a Class?

A class is a **blueprint or specification** from which objects can be created. One architect's blueprint can be used to build many houses; one Student class can create many Student objects.

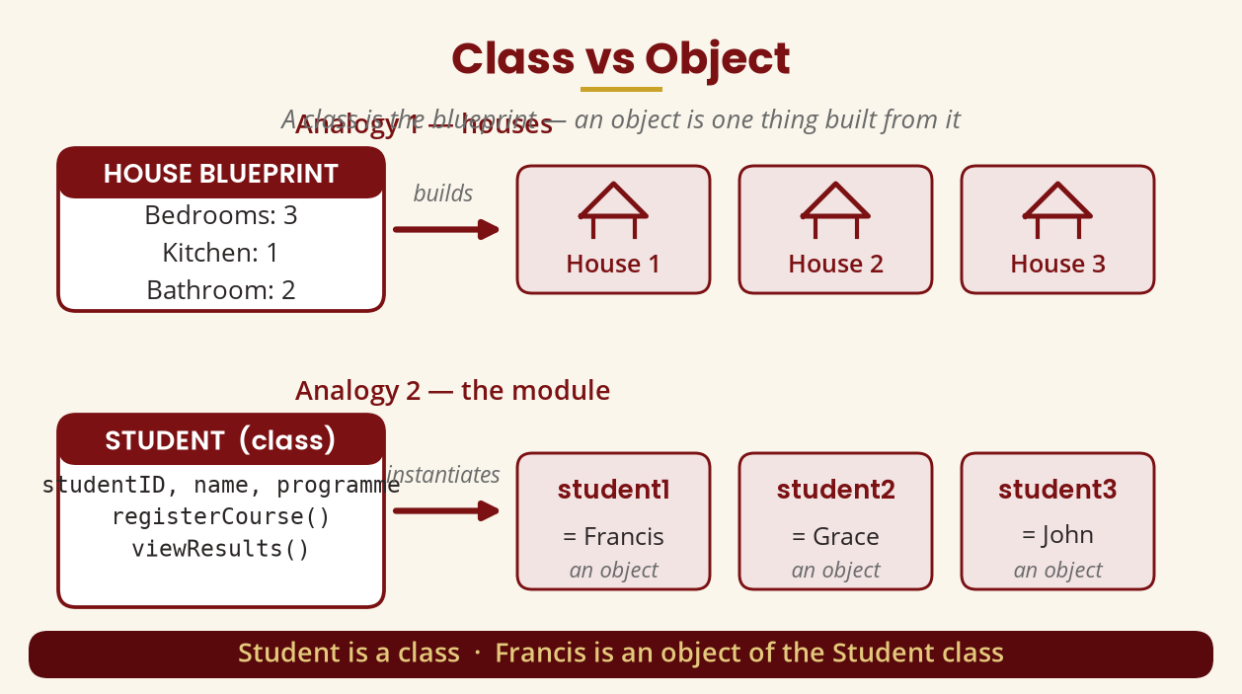


*Figure 10 — Class vs object: the blueprint analogy*

Therefore: **Student is a class; Francis is an object of the Student class.**

## 7.4 Why Object Technology Is Useful in Business

Organisations deal with real-world entities all the time — a bank deals with customers, accounts, transactions, loans and employees; a hospital with patients, doctors, appointments and prescriptions; a university with students, lecturers, courses, programmes and payments. Object-oriented analysis lets analysts represent these important concepts directly in the software model.

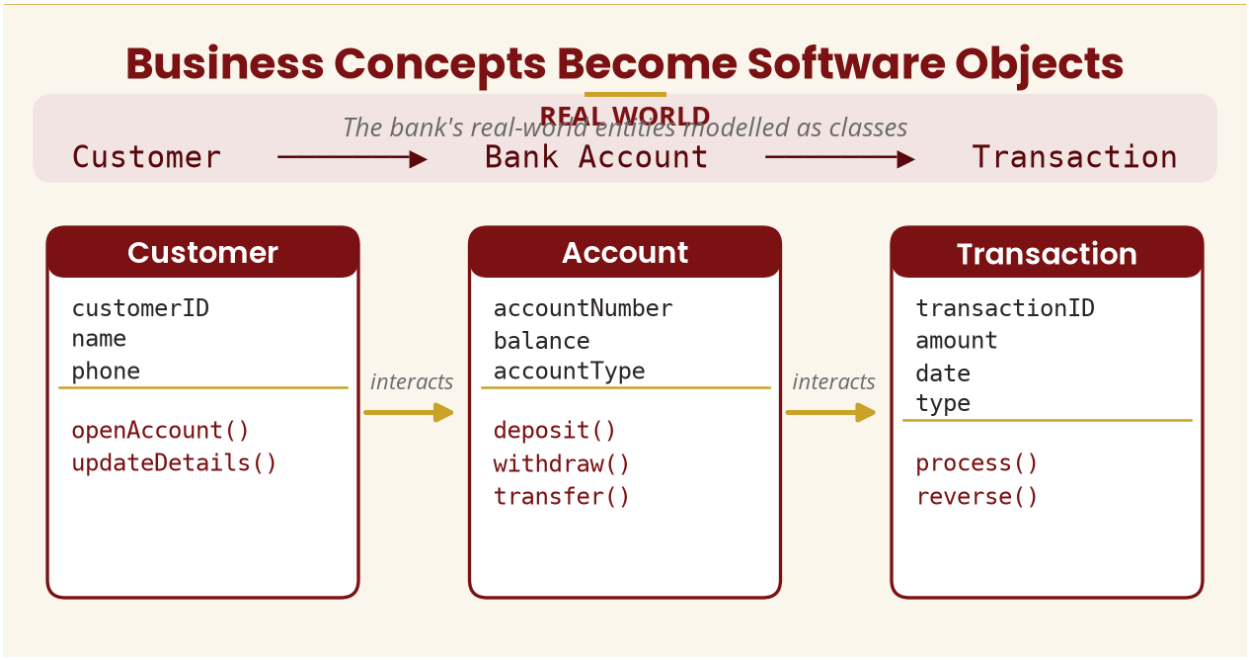


*Figure 11 — The bank's real-world concepts modelled as classes*

**COMMON MISTAKE**

An object does not have to be physically tangible. **Registration, Invoice, Payment, Account, Order, Appointment, Booking and Transaction** are conceptual/business entities, yet they can still be represented as software objects.

## 7.5 Encapsulation — Introduction

Encapsulation means combining data with the operations that work on that data while controlling how the internal state is accessed. We do not want every part of the application directly writing `balance = -500000`; instead the account object controls how its balance changes.

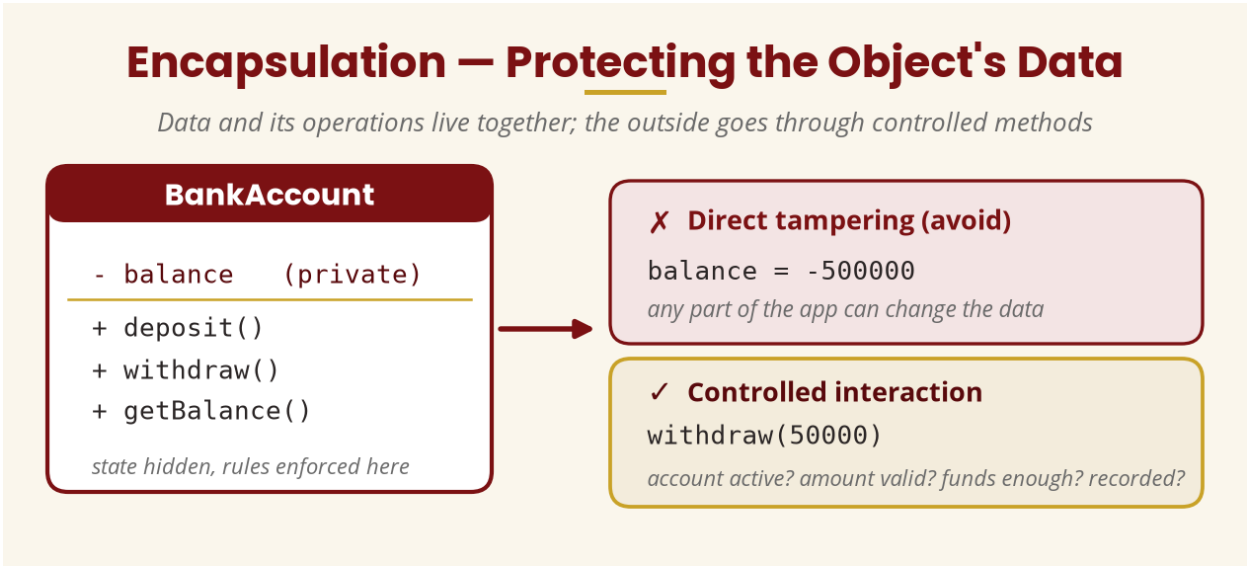


*Figure 12 — Encapsulation protects data behind controlled methods*

## 7.6 Inheritance — Basic Introduction

A general class **User** (userID, name, email, login(), logout()) can have specialised classes **Student, Lecturer and Administrator** that reuse its common characteristics. This is inheritance — studied in detail in Week 6.

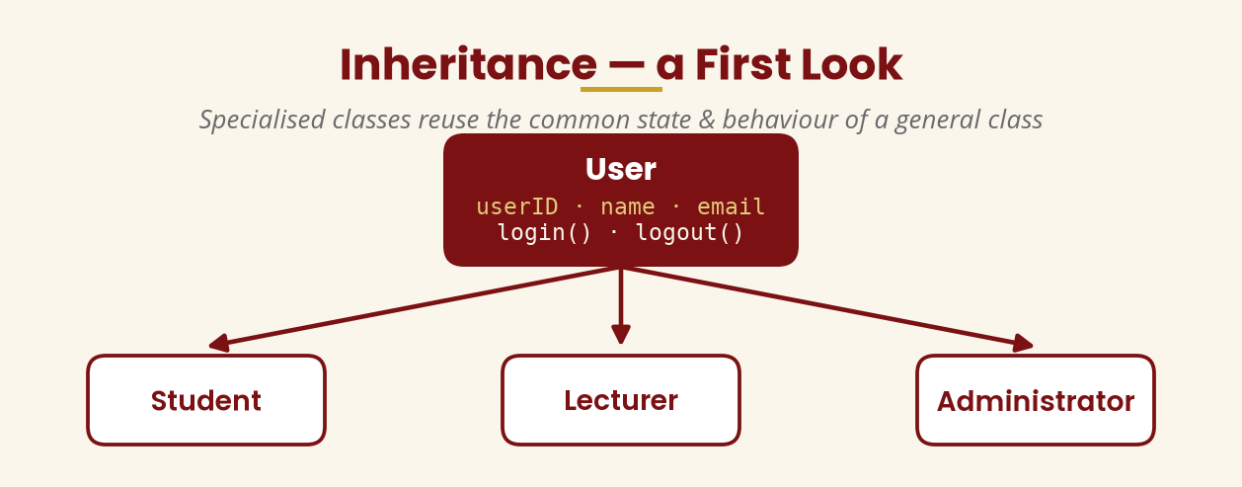


*Figure 13 — Specialised classes reuse common state and behaviour*

## 7.7 Polymorphism — Basic Introduction

Polymorphism means that related objects can respond differently to the same operation. Both Student and Lecturer have `displayDashboard()`, but a Student sees Courses, Fees and Results while a Lecturer sees Courses Taught, Students, Marks and Attendance.

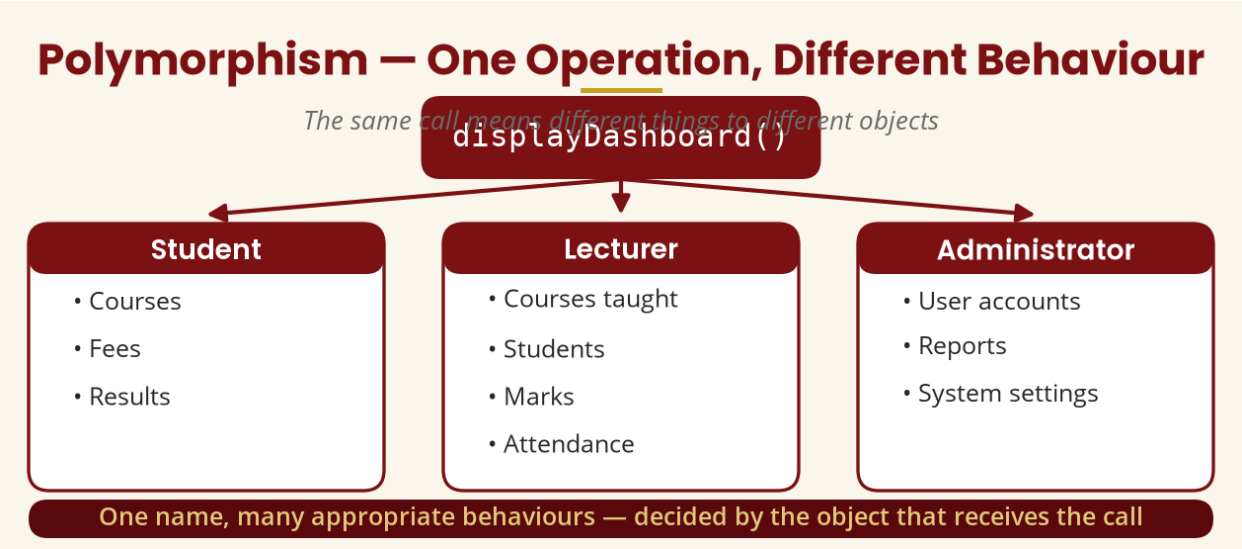


*Figure 14 — One operation, different behaviour per object*

## 7.8 Abstraction — Basic Introduction

Abstraction means focusing on the important characteristics of something while leaving out unnecessary detail. At an ATM the customer sees insert card, enter PIN, select withdrawal, enter amount and receive cash — not the database queries, network protocols, encryption or transaction locking underneath.

## 7.9 The Four OO Principles

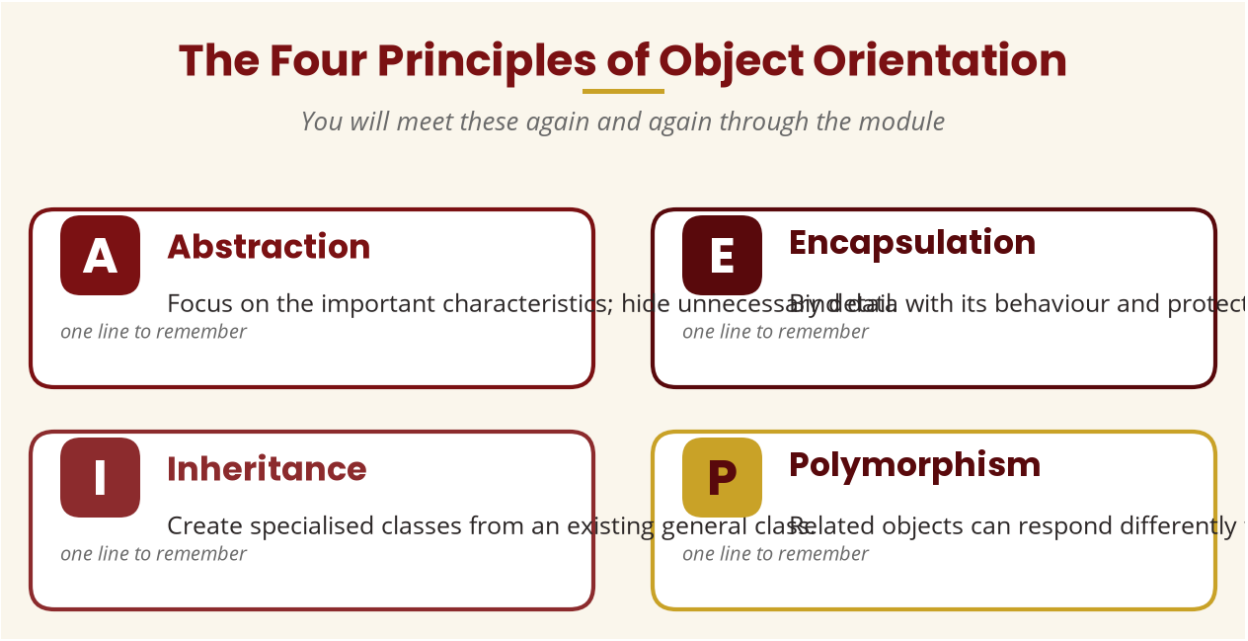


*Figure 15 — Abstraction, encapsulation, inheritance and polymorphism*

## 7.10 Strengths of Object-Oriented Approaches

- **Modularity** — systems divided into related components.
- **Reusability** — existing components and classes can be reused.
- **Maintainability** — well-designed classes isolate change.
- **Encapsulation** — implementation hidden behind controlled interfaces.
- **Extensibility** — systems extended by adding/modifying components.
- **Better representation of complex domains** — Customer, Account, Order, Student, Course appear naturally.
- **Multiple levels of abstraction** — from high-level concepts to detailed designs.

## 7.11 Limitations and Challenges

- **Complexity** — large systems can contain thousands of classes; poor design becomes hard to understand.
- **Learning curve** — objects, classes, relationships, inheritance, interfaces, polymorphism, encapsulation, design principles.
- **Poor modelling adds complexity** — not every noun should become a class (StudentName, StudentAge, StudentChair would be ridiculous).
- **Inheritance can be misused** — deep hierarchies simply because inheritance exists.
- **Initial modelling effort** — careful analysis first, to reduce later misunderstandings.

## 7.12 OOA vs OOD vs OOP

Students often confuse these three stages:

**Object-Oriented Analysis (OOA)** — understanding the problem and modelling **what** the system needs.

**Object-Oriented Design (OOD)** — deciding **how** the software will satisfy those requirements.
**Object-Oriented Programming (OOP)** — **implementing** the design in a language (Java, C#, C++, Python, Kotlin, TypeScript).

> **IMPORTANT**
>
> "I know Java, so I can do OOAD" is not necessarily true. Programming knowledge helps, but OOAD requires understanding business problems, communicating with stakeholders, identifying requirements and domain concepts, modelling relationships and evaluating alternative designs. **A person can be an excellent programmer but a poor analyst.**

# 8. Case Study & Worked Example

## 8.1 Module Case Study — University Registration

Throughout the module we use a continuing example: a **University Student Management and Registration System** replacing a manual process (collect forms, fill details, select courses, obtain approvals, visit finance, submit, wait for data entry).

Problems: long queues, duplicate records, lost forms, incorrect registration, slow reporting, difficulty checking prerequisites and limited visibility of status. The business objective is to *improve the efficiency, accuracy, transparency and accessibility of registration*; the need is *an integrated student registration process*; a potential solution is *an online student management and registration system*.

Candidate objects identified so far: Student, Course, Programme, Department, Lecturer, Registration, Payment, Result, User, Notification. These are candidates only — deciding which become final classes belongs to analysis and design (Week 5).

## 8.2 Worked Example: ATM System

Applying Week 1 concepts to an ATM: the business problem is that customers must visit branches for simple transactions; the need is convenient self-service banking; the objective is 24/7 access with reduced branch workload; the solution is an ATM system.

**Stakeholders:** Customer, Bank, Bank employee, Security administrator, Payment network.
**Functional needs:** authenticate customers, show balance, allow withdrawal/deposits/PIN changes, print receipts.
**Non-functional needs:** secure, reliable, responsive, available 24/7, protect customer information.
**Potential objects:** Customer, Card, Account, Transaction, ATM, Receipt.

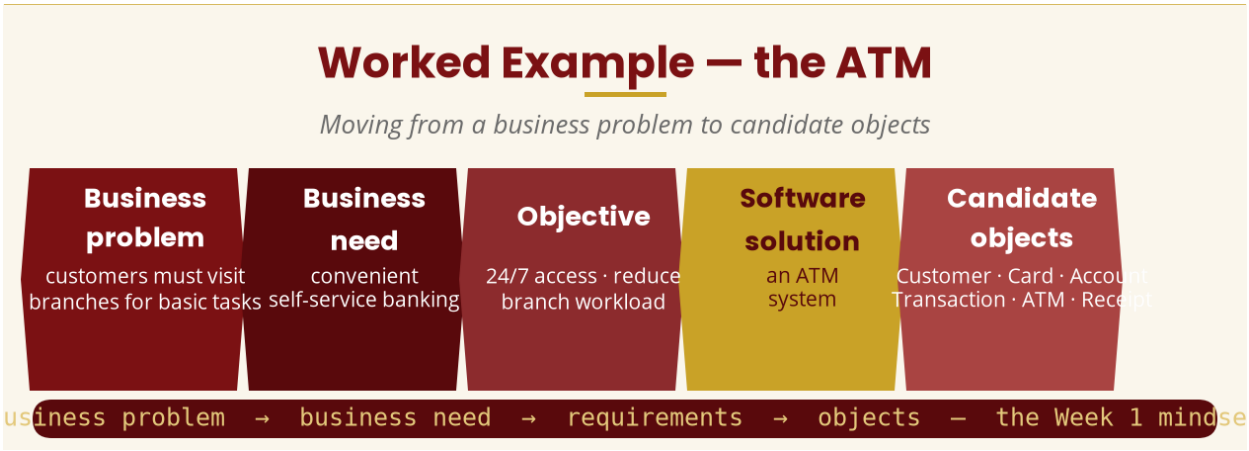


*Figure 16 — The ATM: from business problem to candidate objects*

# 9. Practical & Tutorial Activities

## 9.1 Practical — Identify Business Needs and Candidate Objects

**Scenario:** A college uses a manual library system. Students physically search shelves, fill out borrowing forms, wait for librarians to check records, sometimes lose slips, and cannot easily determine whether a book is available.

**Task 1** — identify at least five business problems.
**Task 2** — identify at least three business needs.
**Task 3** — identify at least six stakeholders.
**Task 4** — write at least eight functional requirements.
**Task 5** — write at least five non-functional requirements.
**Task 6** — identify at least eight candidate objects (e.g. Student, Book, Librarian, Library, Borrowing, Return, Fine, Reservation) and explain why each was selected.

## 9.2 Tutorial — Group Exercise

Each group selects one system — mobile money, hospital management, online shopping, hotel booking or university registration — and produces:

A. Business problem  B. Business need  C. Business objective  D. Stakeholders
E. Functional needs  F. Non-functional needs  G. Business rules  H. Candidate objects

Each group presents its findings to the class.

# 10. Independent Learning (10 Hours)

- **Read** — business needs, requirements, functional/non-functional requirements, stakeholders, object technology.
- **Observe** — choose a system you use regularly (mobile banking, mobile money, registration, supermarket, hospital, library) and write its problem, objectives, needs, stakeholders, functional/non-functional requirements, business rules and possible objects.
- **Object observation** — identify 10 real-world objects; for each write its state and behaviour (e.g. Phone: battery, model, volume — call, charge, restart).

| Object | State | Behaviour |
|---|---|---|
| Phone | battery, model, volume | call, charge, restart |
| Student | name, programme, year | register, study, graduate |
| Car | speed, fuel, gear | start, accelerate, brake |

## 11. Common Student Mistakes

**Mistake 1 — "Business need means software requirement."**

A business need describes what the organisation needs to achieve; a software requirement describes what the system must provide to support that need.

**Mistake 2 — "Every business problem requires software."**

Some problems are organisational, financial, human-resource, procedural or infrastructural rather than technological.

**Mistake 3 — "Functional means important; non-functional means unimportant."**

Both are important. Functional requirements describe behaviour; non-functional requirements describe qualities and constraints.

**Mistake 4 — "An object must be a physical thing."**

Software can model conceptual entities such as Payment, Registration, Order and Transaction.

**Mistake 5 — "A class and an object are the same thing."**

A class defines a type/blueprint; an object is a particular instance.

**Mistake 6 — "OOAD means programming in Java."**

OOAD is about analysing and designing with object concepts; programming is the implementation stage.

**Mistake 7 — "Create classes from every noun immediately."**

Nouns give candidate concepts, but analysis decides which concepts actually belong in the model.

## 12. Week 1 Summary

The table below condenses Week 1 into its essential definitions:

| Term | Definition |
|---|---|
| Business Need | A problem, opportunity or desired improvement an organisation must address. |
| Business Objective | The result the organisation wants to achieve. |
| Stakeholder | A person, group or organisation interested in or affected by the system. |
| Functional Requirement | What the system should do. |
| Non-Functional Requirement | Qualities, constraints or conditions under which the system operates. |
| Business Rule | A policy or condition that governs how the organisation operates. |
| Object | A software entity with identity, state and behaviour. |
| Class | A specification/blueprint from which objects are created. |
| Object Technology | Representing a problem domain using objects, state, behaviour and relationships. |

### 12.1 The Most Important Concept of Week 1

An organisation has a **business objective** □ a **business problem** may prevent it □ the organisation identifies a **business need** □ **stakeholders** help identify what is required □ **requirements** are documented □ analysts study the **problem domain** □ important entities and their behaviour are represented with **object-oriented concepts** □ these models become the foundation for **design and implementation**.

**That is the foundation upon which the remaining 15 weeks will build.**

## 13. Revision Questions

### Short-answer questions

- Define a business need.
- What is a business problem?
- Differentiate between a business problem and a business need.
- What is a business objective?
- Define a stakeholder.
- What is a functional requirement?
- What is a non-functional requirement?
- What is a business rule?
- Define object technology.
- What is an object?

- What is a class?
- Differentiate between a class and an object.
- What is object state?
- What is object behaviour?
- Explain encapsulation.
- What is abstraction?
- What is inheritance?
- What is polymorphism?
- Give four advantages of object-oriented approaches.
- Give three limitations/challenges of object-oriented approaches.

### Application questions

**Question 1.** A hospital has patients waiting several hours before being attended to. Identify: (a) the business problem, (b) two business needs, (c) two business objectives, (d) five stakeholders, (e) five functional requirements, (f) five non-functional requirements, (g) five candidate objects.

**Question 2.** A university wants an online examination management system. Identify: (a) the business need, (b) stakeholders, (c) functional requirements, (d) non-functional requirements, (e) business rules, (f) candidate objects.

**Question 3.** Explain the difference between business need  requirement  software solution, using a real-world example.

### Examination-style question

**EXAM QUESTION**

"A retail organisation currently records sales using handwritten receipts. Management wants to introduce a computerised sales management system. As an object-oriented systems analyst, explain how you would move from understanding the business need to identifying potential objects for the proposed system."

A strong answer discusses: current business problem, business objectives, business needs, stakeholders, requirements elicitation, functional requirements, non-functional requirements, business rules, candidate objects, and the state and behaviour of those candidate objects.

## 14. References and Further Reading

### Standards and authoritative references

- International Organization for Standardization. (2018). *ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering.* ISO. (Current edition; reviewed and confirmed 2024.)
- Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1.* OMG.

## Object-oriented concepts

- Oracle. *Object-Oriented Programming Concepts.* Oracle documentation on objects, classes, inheritance, encapsulation and related concepts.
- Microsoft. *Object-oriented programming (C#).* Microsoft Learn, updated 2025 — abstraction, encapsulation, inheritance and polymorphism.

## Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling.*
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML.*
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide.*
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design.*
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements.*
- Wixom, B., & Dennis, A. (2020). *Systems Analysis and Design: An Object-Oriented Approach with UML.*